Faculty of Computers and Artificial Intelligence
Cairo University
Natural Language Processing – Spring 2026

Project: Text Classification using Text Representation Schemes [10 + 2 Marks]

Submission Rules:

1. Students will form teams of 7 students (Can be from any group).
2. Deadline of submission is – 2 May 2025 @ 11:59 PM
3. Submission will be on google classroom.
4. No late submission is allowed.
5. No submission through e-mails.
6. Submit one compressed folder having two files that follow the naming convention (ID1_ID2_ID3_ID4_ID5_ID6_ID7.***ipynb and*** ID1_ID2_ID3_ID4_ID5_ID6_ID7.***pdf*** *both in a zipped folder named* ID1_ID2_ID3_ID4_ID5_ID6_ID7)
7. **Penalty will be applied to the following cases:**
   a. Working in a team with less than three students (-0.5)
   b. Not following the naming convention for the three items: folder name, source code and report. (-0.5)
   c. **Any minor similarity between your code and AI-generated code or any other source, will lead to receiving ZERO.**
8. **Read the requirements very carefully and make sure to implement them as required to avoid losing grades.**
9. **You are obligated to use the libraries mentioned in each requirement.**
10. **The report has about 20% of the grade in part II, so make sure to include in it all what is required, even if you output these requirements in your notebook.**

Part I – Reviews Sentiment Classification using TF-IDF

Dataset: Amazon reviews dataset; This dataset was created from the scraped reviews from products in Amazon, it contains over 17000+ records attached with the assignment document named "*amazon_reviews.csv*". The classes are three in number namely, Negative Reviews, Neutral Reviews, Positive Reviews.

Requirements:

1. Reviews pre-processing: remove stop words using NLTK.
2. Label encoding: mapping labels (positive, negative, neutral) into numerical labels using **Sci-kit learn**.

3. Vectorization: Apply TF-IDF vectorizer to the reviews dataset to get the feature vectors corresponding to the reviews using **Sci-kit learn.**
4. Data Splitting: use the splitting ratio 80% training and 20% testing using **Sci-kit learn**.
5. Text classification: train the following models **Sci-kit learn** SVM and logistic regression using **Sci-kit learn.**
6. Classification report: output the classification report for each model using *(**hint:** you will use Sci-kit learn built-in classification report).*
7. [Bonus: 1 Mark] User Input: Allow the user to enter a new review, apply vectorization then predict the label of the new review. *(**hint:** you will use the same vectorizer used for transforming the dataset by passing the new review to a function called transform).*

Part II – Tweets Emotion Classification using Word Embedding

Dataset: Twitter Emotion Dataset; it contains Over 400000+ Twitter messages attached with the assignment document "*twitter_emotion.csv*", each entry features a text segment extracted from Twitter, accompanied by a corresponding label denoting the predominant emotion conveyed by the message. The emotions are categorized into six distinct classes: sadness (0), joy (1), love (2), anger (3), fear (4), and surprise (5).

Requirements:

1. Tweets pre-processing: convert tweets into padded sequences and build dictionary of (word, index) using **keras** tokenizer with a max. number of words = 100,000.
2. Word embeddings: You will apply two types of embeddings:
    a. Pre-trained word embeddings model: you will download glove-twitter-50 word vectors/embeddings using **gensim** downloader API and use it to construct your embedding matrix that will be used in **keras** embedding layer.
    b. Your trained word2vec model word embeddings: you will use the word vectors obtained from training your own word2vec model to construct your embedding matrix that will be used **keras** embedding layer. (check the section below of " how to train a word2vec model?).
3. Data preparation and splitting:
    a. Apply one-hot encoding for the integer labels using **keras.**
    b. Try different splitting ratios: (70% training and 30% testing, 80% training and 20% testing).
4. Build CNN Models: you will build two CNN models; the first model uses pre-trained word embeddings, and the second model uses word2vec trained model, the model components are:
    a. Embedding layer.
    b. Two 1D convolutional layers.
    c. Output layer.
5. [Bonus: 1 Mark] User input: classify an input message using the model with higher accuracy.

Report: Provide a report that shows model summary (generated using **keras**) of each model and the best hyperparameters for each model (splitting ratio, sequence padding length …). Provide a table that compares between accuracy of each model at different sequence padding lengths (500, 700, 1000…).

How to train a word2vec model?

6. Tweets pre-processing: Apply tokenization to the list of tweets, such that each tweet is transformed into a list of tokens using **gensim** pre-processing utility.
7. Word2Vec Model Training: train your own word2vec model using the tokenized tweets from the previous step to obtain word embeddings using **gensim**.